



TU Clausthal

Master's Thesis

by

Yu Zhang

**SimTutor: A Telemetry-Grounded Tutoring
Runtime with
Structured Visual Fact Extraction for Procedural
Learning**

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Prof. Dr. Rüdiger Ehlers**

9th May 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 9th May 2026

Location, Date

Yu Zhang

Abstract

Zusammenfassung

Acknowledgements

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Thesis Vision and Scope	1
1.3	Current Implementation Scope	1
1.4	Contributions	1
1.5	Thesis Structure	1
2	Background and Related Work	2
2.1	Procedural Tutoring and Intelligent Assistance	2
2.2	Grounded LLM Systems and Retrieval-Augmented Help	2
2.3	Vision-Language Model Adaptation for Structured Outputs	2
2.4	Simulator-Based Training and Instrumentation	2
2.5	Human-Subject Evaluation in Procedural Learning	2
2.6	Positioning of This Work	2
2.7	The F/A-18C Cold-Start Task	2
2.8	Visual Fact Ontology	2
3	System Design	3
3.1	Design Goals and Requirements	3
3.2	Overall Architecture	5
3.3	Separation of Core Tutoring Logic and Simulator Adapters	6
3.4	Data Flow	7
3.4.1	Observation	7
3.4.2	Tutor Request	7
3.4.3	Context Assembly	8
3.4.4	Model Inference (Optional)	8
3.4.5	Response Validation and Mapping	8
3.4.6	Action Execution	9

3.4.7	Event Logging	9
3.5	Telemetry Processing	9
3.6	Procedure Engine and Gating Rule Engine	10
3.6.1	Procedure Engine	10
3.6.2	Gating Rule Engine	10
3.6.3	Deterministic Step Inference	10
3.6.4	Procedure Pack Structure	10
3.7	Simulator Adapter Layer	11
3.7.1	Telemetry Input Adapter	11
3.7.2	Overlay Output Adapter	11
3.7.3	Vision Input Adapter	11
3.7.4	Lua-Side Integration	11
3.7.5	Planned and Partially Implemented Adapters	12
3.8	Event Logging and Replay	12
3.8.1	Event Model	12
3.8.2	JSONL Event Store	12
3.8.3	Replay and Validation	12
3.8.4	Scoring and Interaction Metrics	12
3.8.5	Current Limitations	12
3.9	Optional VLM Adaptation for Visual State Understanding	13
3.9.1	Role of the VLM Component	13
3.9.2	Vision Fact Ontology	13
3.9.3	VLM Integration in the Help Cycle	13
3.9.4	VLM Adaptation Pipeline	14
3.9.5	Distinction from Rule-Based State Gating	14
3.10	Differences from the Original Proposal	14
3.10.1	From VR-First to Desktop-First Runtime	14
3.10.2	Added VLM Adaptation Pipeline	14
3.10.3	Expanded Telemetry Grounding	15
3.10.4	Added Procedural, Replay, and Benchmark Infrastructure	15
3.10.5	Postponed Human-Subject Evaluation	15
3.10.6	Summary of Design Evolution	15
4	Methodology	17
4.1	Procedure Runtime and Telemetry Grounding	17

4.2	Knowledge Grounding and Source Policy	17
4.3	Visual Fact Data Pipeline	17
4.4	Dataset Curation	17
4.5	LoRA Fine-Tuning Configuration	17
4.6	Benchmark Protocol	17
5	Implementation	18
5.1	Technology Stack	18
5.2	Core Domain Logic	18
5.3	Telemetry and DCS Integration	18
5.4	Structured Help Generation	18
5.5	VLM Visual Fact Extraction	18
5.6	Replay and Logging Infrastructure	18
5.7	CLI and Runtime Entrypoints	18
6	Evaluation	19
6.1	Current Technical Results	19
6.1.1	Dataset and Training Summary	19
6.1.2	Qwen3.5 8-Fact V1 Baseline	19
6.1.3	Qwen3.5 13-Fact V2 Results	19
6.1.4	Gemma-4 13-Fact V2 Comparison	19
6.1.5	Error Analysis	19
6.2	Planned Runtime Data Collection and Human-Subject Evaluation	19
6.2.1	Original Evaluation Vision	19
6.2.2	Planned Evaluation Metrics	19
6.2.3	Required Runtime Data Collection Upgrades	19
6.2.4	Planned Experiment Design	19
6.2.5	Known Threats to Validity	19
7	Discussion	20
7.1	Why the Project Shifted from VR-First to Telemetry/VLM-Heavy	20
7.2	What Current Benchmarks Do and Do Not Prove	20
7.3	Ontology Evolution: 8-Fact to 13-Fact	20
7.4	Why Human-Subject Evidence Remains Essential	20
7.5	Risks in Interpreting Benchmark Success as Learning Success	20
7.6	Limitations	20

8 Conclusion	21
8.1 Summary of Completed Work	21
8.2 Future Work	21
References	22
9 Appendix	23

List of Figures

3.1	SimTutor system architecture. The core layer contains simulator-agnostic domain logic. The ports layer defines abstract interfaces. The adapters layer provides concrete implementations for DCS, model providers, knowledge sources, and the vision pipeline. Procedure packs supply declarative configuration.	6
3.2	Main runtime data flow through the help cycle.	7

List of Tables

3.1 Design evolution from proposal to implemented system. 16

1 Introduction

1.1 Motivation

1.2 Thesis Vision and Scope

1.3 Current Implementation Scope

1.4 Contributions

1.5 Thesis Structure

2 Background and Related Work

2.1 Procedural Tutoring and Intelligent Assistance

2.2 Grounded LLM Systems and Retrieval-Augmented Help

2.3 Vision-Language Model Adaptation for Structured Outputs

2.4 Simulator-Based Training and Instrumentation

2.5 Human-Subject Evaluation in Procedural Learning

2.6 Positioning of This Work

2.7 The F/A-18C Cold-Start Task

2.8 Visual Fact Ontology

3 System Design

This chapter describes the design of SimTutor, a telemetry-grounded tutoring runtime for procedural training in the DCS F/A-18C cold-start task. Section 3.1 derives the design goals from the thesis problem statement. Section 3.2 presents the overall architecture and the separation of core tutoring logic from simulator-specific adapters. Section 3.4 describes the runtime data flow through the main help cycle. Sections 3.5 and 3.6 cover the telemetry processing pipeline and the procedure and gating rule engines. Section 3.7 details the simulator adapter layer and Lua-side integration. Section 3.8 describes the event logging, replay, and scoring infrastructure. Section 3.9 presents the optional VLM adaptation component for visual state understanding. Section 3.10 concludes with a systematic comparison to the original thesis proposal, documenting how the design evolved during implementation.

3.1 Design Goals and Requirements

The thesis addresses the problem of providing grounded, context-aware procedural guidance to a learner operating a complex simulator. In the target scenario—the F/A-18C cold-start procedure in DCS World—the learner must execute a fixed sequence of 25 steps (S01–S25) spanning six phases, from battery power-up through engine start, avionics configuration, flight-control checks, and taxi preparation. Each step has specific pre-conditions that must be satisfied before execution and completion conditions that must be met before advancing. Mistakes include omissions, order errors, parameter-setting errors, and state violations—categories formalised in the error taxonomy described in Chapter 4.

From this problem, we derive the following design goals:

G1 – Simulator-based procedural tutoring. The system must observe the learner’s actions through simulator state, determine which procedural step the learner is currently attempting, and provide timely, grounded guidance when the learner is stuck

or requests help. The tutoring output must reference concrete cockpit elements and cite evidence from telemetry, documentation, or visual observations.

- G2 – State-aware step guidance.** The system must track procedure progress through a formal state machine. It must evaluate preconditions before allowing a step to become active and verify completion conditions before advancing. Guidance must be conditional on the current simulator state, not merely on step order.
- G3 – Separation between tutoring logic and simulator-specific integration.** The core tutoring logic—procedure tracking, gating, scoring, and help generation—must be independent of any particular simulator, model provider, or knowledge source. Simulator-specific code, model-provider code, and knowledge-retrieval code must be isolated behind stable interfaces so that the core can be tested without a running simulator and can be ported to other simulators or training domains.
- G4 – Replayability and inspectability.** Every interaction between the learner, the tutoring system, and the simulator must be recorded as structured, timestamped events. These event logs must support offline replay for debugging, automated scoring against a procedure pack, and post-hoc analysis of learner behaviour. They must also support replay of telemetry captures and benchmark traces for the VLM adaptation pipeline.
- G5 – Optional foundation-model and VLM support.** The system must be able to operate with or without a large language model. When a model is available, it may provide richer diagnoses and explanations. When unavailable or when its output fails validation, the system must degrade gracefully to deterministic, rule-based fallback behaviour. Vision-language model (VLM) support for visual fact extraction from cockpit screenshots must be an optional, independently operable component.
- G6 – Extensibility to different simulators and procedures.** The procedure definition, UI target mapping, telemetry mapping, error taxonomy, and knowledge source policy must be expressed as declarative configuration (YAML files) rather than hard-coded in source code. Adding a new procedure or adapting the system to a different simulator should require only new configuration artifacts, not changes to the core tutoring logic.

These goals directly inform the architectural decisions described in the following sections.

3.2 Overall Architecture

SimTutor follows a ports-and-adapters (hexagonal) architecture. The system is organised into three main layers, each with clearly defined responsibilities and dependency directions.

Core layer (core/). The core contains all domain logic that is independent of any particular simulator, model provider, or external service. It defines the data contracts for runtime messaging (Section 3.4), the procedure state machine and gating rule engine (Section 3.6), the scoring and interaction-metrics engines, the vision-fact ontology and aggregation rules, the BM25 retrieval implementation, the HelpResponse schema builder, and the security and audit infrastructure. The core depends on nothing outside itself except standard Python libraries and the abstract interfaces defined in the ports layer.

Ports layer (ports/). The ports layer defines stable, technology-agnostic Python protocols for the four external dependencies the core may need at runtime: model reasoning (`ModelPort`), knowledge retrieval (`KnowledgePort`), vision input (`VisionPort`), and telemetry input (`TelemetryPort`). These interfaces are the only dependency injection points between the core and the outside world.

Adapters layer (adapters/). The adapters layer contains all simulator-specific, provider-specific, and infrastructure-specific code. It includes DCS-BIOS telemetry receivers, the telemetry enrichment and delta-sanitisation pipeline, DCS overlay senders, OpenAI-compatible model clients (with and without multimodal support), Ollama fallback clients, a deterministic model stub for testing, a local BM25 knowledge adapter, the vision-fact extraction runtime, and the response-mapping and action-execution components that bridge model output to runtime behaviour. Adapters depend on the ports they implement and on the core types they produce and consume; they never introduce reverse dependencies into the core.

The three layers are complemented by two further structural elements:

Procedure packs (packs/). A procedure pack is a directory of declarative YAML configuration files that fully describe one procedural task. The pack defines the step list, preconditions and completion conditions (gating rules), UI targets and their simulator-specific element identifiers, telemetry variable mappings, error taxonomy categories and scoring weights, delta-sanitisation policy, vision-fact ontology with

step bindings, and vision layout description. The current repository contains one pack: `fa18c_startup`, covering the full F/A-18C cold-start procedure.

CLI and runtime entrypoints (`simtutor/`, `live_dcs.py`). The `simtutor` package provides a command-line interface for running mock scenarios, replaying and validating event logs, scoring runs, recording VLM predictions for benchmarking, and replaying DCS-BIOS raw captures. The `live_dcs.py` module orchestrates the full live runtime loop: it wires together a DCS-BIOS receiver, the telemetry enrichment pipeline, knowledge retrieval, model-based help generation, optional VLM visual fact extraction, overlay action execution, and event logging into a single long-running process.

Figure 3.1 shows the high-level component diagram.

Figure 3.1: SimTutor system architecture. The core layer contains simulator-agnostic domain logic. The ports layer defines abstract interfaces. The adapters layer provides concrete implementations for DCS, model providers, knowledge sources, and the vision pipeline. Procedure packs supply declarative configuration.

3.3 Separation of Core Tutoring Logic and Simulator Adapters

The separation between simulator-independent core logic and simulator-specific adapters serves four concrete purposes:

1. **Testability.** The core logic can be tested with mock adapters and deterministic scenarios without requiring a running DCS instance, a GPU, or network access to a model server. The repository includes a `MockEnvAdapter` that replays pre-recorded observation sequences and a `ModelStub` that returns deterministic responses, enabling fast, reproducible unit and integration tests.
2. **Maintainability.** When a simulator API changes, only the affected adapter needs to be updated; the core remains unchanged. When a model provider changes (e.g., from Ollama to an OpenAI-compatible vLLM server), only the model adapter configuration is affected. Three providers (`openai_compat`, `ollama`, `stub`) are supported through a single configuration interface.

3. **Transferability.** Porting the system to a new simulator or aircraft requires writing new telemetry and overlay adapters and creating new procedure packs. The procedural reasoning, gating, scoring, and help-generation infrastructure can be reused as-is. The port interfaces are intentionally minimal: a telemetry port requires `start`, `poll`, and `stop` methods; an overlay port requires only the ability to send highlight, clear, and pulse commands to named cockpit elements.
4. **Independent evolution of the VLM pipeline.** The VLM visual-fact extraction component communicates with the tutoring runtime through the vision port and the vision-fact observation schema, not through direct coupling. The VLM pipeline can be developed, trained, and benchmarked independently of the tutoring runtime. A runtime deployment can include the VLM component, omit it entirely, or replace it with a different perception backend without affecting the tutoring logic.

3.4 Data Flow

The runtime data flow follows a structured pipeline from simulator observation to tutoring output and logging. Figure 3.2 provides a visual summary.

Figure 3.2: Main runtime data flow through the help cycle.

3.4.1 Observation

The simulator adapter—in the current implementation, a DCS-BIOS UDP receiver—produces a stream of *observations*: versioned, timestamped data objects containing raw simulator state and, after enrichment, a set of normalised state variables.

3.4.2 Tutor Request

When the learner triggers help—via a hotkey, a UDP message, or a stall-detection heuristic—a *tutor request* is created. The request carries an intent (`help`), an optional free-text message, a reference to the current observation, and a context dictionary populated by the runtime orchestrator.

3.4.3 Context Assembly

The runtime orchestrator assembles a context for the help cycle by gathering:

- **State variables** (`vars`): normalised key–value pairs from the most recent telemetry observation.
- **Recent deltas** (`recent_deltas`): a sanitised, aggregated summary of recent state changes.
- **Candidate steps** (`candidate_steps`): eligible step identifiers from the procedure pack.
- **Deterministic step hint** (`deterministic_step_hint`): a locally computed fallback suggestion from rule-based inference, used both as prompt guidance and as fallback when model output is unavailable.
- **Overlay target allowlist** (`overlay_target_allowlist`): the set of UI elements the model is permitted to reference.
- **Knowledge snippets** (`rag_topk`, optional): top- k BM25 retrieval results, included only when a knowledge index exists.

3.4.4 Model Inference (Optional)

If a ModelPort implementation is available, the assembled context is formatted into a prompt and sent to the language model. The model returns a structured JSON object conforming to the HelpResponse schema, which is generated dynamically at runtime from the procedure pack, UI map, and taxonomy.

3.4.5 Response Validation and Mapping

The raw model output passes through a validation pipeline: (1) JSON extraction with minimal repair, (2) schema validation, (3) allowlist checks on diagnosis step IDs, next step IDs, and overlay targets, (4) evidence-grounding validation. If validation succeeds, the help object is mapped to a **TutorResponse** with overlay actions resolved through the UI map. If validation fails, or if no model is available, the system enters deterministic fallback.

3.4.6 Action Execution

The tutor response may contain overlay actions (highlight, clear, or pulse) targeting specific cockpit elements. The action executor validates each action against the UI target allowlist, enforces a limit on concurrent highlights (default: one), and dispatches the command to the simulator via UDP. Auto-click and control-injection actions are forbidden by design.

3.4.7 Event Logging

Every stage of the help cycle is recorded as a versioned, timestamped **Event** object and appended to a JSONL event log, which serves as the authoritative record for all downstream analysis, replay, and scoring.

3.5 Telemetry Processing

The telemetry processing subsystem transforms raw simulator data into the normalised state representation through four stages:

1. **Raw ingestion.** The DCS-BIOS receiver listens on a UDP socket for binary DCS-BIOS export frames, decoding each into a structured JSON object with wall-clock timestamps and monotonically increasing sequence numbers.
2. **Variable resolution.** A telemetry map specifies how raw BIOS fields are projected onto normalised state variables. The variable resolver produces a flat dictionary of key–value pairs and tracks unavailable or unknown sources in a `vars_source_missing` set, enabling downstream components to distinguish “the value is zero” from “the value is unknown.”
3. **Delta sanitisation.** A configurable policy (thresholds, debounce windows, per-variable filtering rules) suppresses spurious changes caused by switch bounce, sensor drift, and UDP out-of-order delivery.
4. **Delta aggregation.** Sanitised deltas are accumulated over a configurable time window and compressed into a compact summary for the prompt context.

The telemetry pipeline runs continuously, independent of the help cycle.

3.6 Procedure Engine and Gating Rule Engine

3.6.1 Procedure Engine

The procedure engine maintains a finite-state machine over the ordered sequence of steps. Each step is in one of four states: `pending`, `active` (at most one), `done`, or `blocked`. Operations include `activate_next`, `complete_active`, `block_active`, `rewind`, and `check_timeout`. All transitions are recorded as internal events and propagated to the event log.

3.6.2 Gating Rule Engine

The gating rule engine evaluates precondition gates and completion gates using a v1 DSL with four operators: `var_gte`, `arg_in_range`, `flag_true`, and `time_since`. The evaluator includes explicit unknown-value handling: `missing`, `source-missing`, or `sentinel-value` variables cause rule failure with a diagnostic reason rather than silent treatment as `false`. Rules short-circuit on first failure; the failure index and reason feed into the deterministic step hint.

3.6.3 Deterministic Step Inference

When no model is available or model output fails validation, the system falls back to deterministic step inference, which examines the active step, unsatisfied gating rules, and recent UI targets to produce a safe text hint. By default, no overlay highlights are sent in fallback mode unless a local rule can prove a unique safe target.

3.6.4 Procedure Pack Structure

The procedure pack is the declarative configuration driving the entire tutoring runtime. Each step is classified by primary evidence source:

- **BIOS-priority steps** (S01, S03–S07, S09–S13, S21): completion evidence primarily in telemetry variables.
- **Vision-priority steps** (S08, S15, S18): require visual confirmation through cockpit displays.
- **Manual or out-of-layout steps** (S02, S14, S16, S17, S19, S20, S22–S25): cannot be fully observed through either telemetry or the current visual layout.

The pack also defines per-step gating rules, UI targets, tutor prompts, and profile overrides (airfield vs. carrier).

3.7 Simulator Adapter Layer

3.7.1 Telemetry Input Adapter

Receives DCS-BIOS state data over UDP. The Python-side `DcsBiosRawReceiver` parses the binary protocol and emits raw frames as JSONL records, which feed the telemetry enrichment pipeline.

3.7.2 Overlay Output Adapter

Sends highlight, clear, and pulse commands to DCS via plain-text UDP messages. A Lua script (`VRHilite.lua`) inside DCS renders graphical overlays on cockpit elements. The adapter maps abstract target names to DCS-internal point codes using the procedure pack's UI map.

3.7.3 Vision Input Adapter

Triggers screenshot captures in DCS via a UDP-based capture trigger. The Lua-side `SimTutor.lua` renders the three primary cockpit displays into a single composite image. The Python-side adapter selects pre-trigger and trigger frames and passes them to the vision-fact extractor.

3.7.4 Lua-Side Integration

Three categories of Lua scripts execute inside DCS:

- **State export** (`Export.lua`): configures DCS-BIOS export.
- **Overlay rendering** (`Hooks/VRHilite.lua`, `Hooks/SimTutorHighlight.lua`): receives and renders highlight commands.
- **SimTutor integration** (`SimTutor/SimTutor.lua`, `SimTutor/SimTutor Function.lua`, `SimTutor/SimTutorDcsBiosHub.lua`): handles capture and composite-panel rendering.

3.7.5 Planned and Partially Implemented Adapters

The current repository includes a `MockEnvAdapter` for deterministic testing without a running simulator. The original thesis proposal envisioned a Meta Quest 3 VR adapter. This adapter has not been implemented; the VR deployment remains planned (Section 6.2).

3.8 Event Logging and Replay

3.8.1 Event Model

All runtime occurrences are represented as `Event` objects with a fixed schema: unique identifier, wall-clock timestamp, session identifier, event kind (`observation`, `tutor_request`, `tutor_response`, `overlay_requested`, `overlay_applied`, `overlay_failed`), typed payload, schema version, and extensible metadata.

3.8.2 JSONL Event Store

Events are persisted in JSONL format via `JsonlEventStore`, supporting append-only writing and full-file loading.

3.8.3 Replay and Validation

A recorded event log can be replayed offline to verify step ordering and state machine consistency. Telemetry logs are independently validated for monotonic sequence numbers and timestamps.

3.8.4 Scoring and Interaction Metrics

The scoring engine counts omissions and state violations with configurable per-category weights and a critical-step multiplier. Interaction metrics characterise learner behaviour: stall time, help requests, LLM triggers, UI click counts, and overlay request/acknowledgement counts.

3.8.5 Current Limitations

The replay infrastructure has not yet been extended to replay full multimodal sessions with synchronised vision frames.

3.9 Optional VLM Adaptation for Visual State Understanding

Visual perception in SimTutor is an *optional, independently operable* subsystem.

3.9.1 Role of the VLM Component

The VLM component extracts structured *visual facts* from cockpit screenshots—intermediate propositions about visible display content. Visual facts are not final tutoring decisions; they provide visual evidence that the tutoring runtime combines with telemetry, procedure state, and documentation. This decomposition enables each component to be developed, tested, and evaluated independently.

3.9.2 Vision Fact Ontology

The current ontology (v2) defines 13 facts spanning three display regions:

- **Left DDI:** `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_v`
- **Right DDI:** `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_v`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD:** `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`.

Each fact has a three-valued state (`seen`, `not_seen`, `uncertain`), a time-to-live, an optional `sticky` flag, and step bindings via `all_of` / `any_of` conditions.

3.9.3 VLM Integration in the Help Cycle

When a help request arrives and the active step is vision-priority (S08, S15, S18), a capture trigger is sent to DCS. Pre-trigger and trigger frames are rendered and sent to the VLM. The returned facts are merged into the vision-fact snapshot (with sticky and TTL semantics) and included in the help-cycle context. If the vision adapter is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only.

3.9.4 VLM Adaptation Pipeline

The VLM used at runtime is fine-tuned. The adaptation pipeline—detailed in Chapter 4—comprises screenshot capture, pre-labeling by a large VLM, human review in Label Studio, bilingual SFT export, LoRA fine-tuning (Unsloth + PEFT + TRL), and base-vs-LoRA benchmarking on independent held-out sessions. This pipeline is a complete, self-contained contribution.

3.9.5 Distinction from Rule-Based State Gating

Rule-based gating evaluates telemetry variables deterministically; it is fast and auditable but limited to telemetry-representable conditions. VLM-based visual fact extraction interprets display content that has no telemetry representation. The two mechanisms are complementary: the gating engine provides the primary procedure-tracking backbone; visual facts supply supplementary evidence for steps whose completion cannot be inferred from telemetry alone.

3.10 Differences from the Original Proposal

The system described in this chapter differs in several important respects from the system envisioned in the original thesis proposal. We document these differences to establish a clear boundary between the original research vision and the current implemented system.

3.10.1 From VR-First to Desktop-First Runtime

The original proposal described a Meta Quest 3 VR tutoring system with in-headset step cards. The current implementation is a desktop/DCS runtime with cockpit overlay highlighting as the primary output modality. This shift occurred because building a robust telemetry-grounded runtime proved a substantial prerequisite for VR deployment, and the desktop configuration provides a more controlled environment for systematic benchmarking. The VR deployment remains a planned next phase (Section 6.2).

3.10.2 Added VLM Adaptation Pipeline

The original proposal envisioned a text-only RAG+LLM tutor. The VLM visual-fact extraction pipeline was added after discovering that several critical procedure steps require visual confirmation not inferable from telemetry alone. The VLM pipeline evolved into

a complete adaptation and benchmarking loop that now constitutes a major technical contribution.

3.10.3 Expanded Telemetry Grounding

The current implementation includes a substantially more elaborate telemetry subsystem: raw DCS-BIOS UDP ingestion, variable resolution with source-missing tracking, delta sanitisation, and delta aggregation. This expansion was driven by the practical realities of working with noisy, high-frequency simulator telemetry.

3.10.4 Added Procedural, Replay, and Benchmark Infrastructure

The original proposal did not describe a formal procedure engine, gating rule engine, error taxonomy, event logging, replay, or automated benchmarking. These were developed during implementation as essential infrastructure for systematic evaluation.

3.10.5 Postponed Human-Subject Evaluation

The original proposal planned a three-condition human-subject study that has not been conducted. The current repository contains the technical infrastructure to support such a study but no participant data. The human-subject evaluation remains the central planned component (Section [6.2](#)).

3.10.6 Summary of Design Evolution

Table [3.1](#) summarises the key differences.

This evolution does not invalidate the original proposal. Rather, the implemented system provides a significantly more robust technical baseline on which the planned VR deployment and human-subject evaluation can be built.

Table 3.1: Design evolution from proposal to implemented system.

Aspect	Original Proposal	Implemented System
Platform	Meta Quest 3 VR	Desktop / DCS
Output modality	In-headset step cards	Cockpit overlay highlights
State input	Read-only DCS state flags	Full DCS-BIOS telemetry with delta sanitisation
Visual perception	None (text-only RAG)	Optional VLM visual-fact extraction (13-fact v2)
Model adaptation	Not addressed	LoRA fine-tuning pipeline (Qwen3.5-9B, Gemma-4-31B)
Procedure representation	Implicit in prompts	Explicit pack-driven configuration with gating rules
Error handling	Not specified	Structured taxonomy, deterministic fallback, safety boundary
Evaluation infrastructure	Not specified	Event logging, replay validation, automated scoring, benchmark pipeline
Human-subject study	Planned as main evaluation	Postponed; infrastructure prepared

4 Methodology

4.1 Procedure Runtime and Telemetry Grounding

4.2 Knowledge Grounding and Source Policy

4.3 Visual Fact Data Pipeline

4.4 Dataset Curation

4.5 LoRA Fine-Tuning Configuration

4.6 Benchmark Protocol

5 Implementation

5.1 Technology Stack

5.2 Core Domain Logic

5.3 Telemetry and DCS Integration

5.4 Structured Help Generation

5.5 VLM Visual Fact Extraction

5.6 Replay and Logging Infrastructure

5.7 CLI and Runtime Entrypoints

6 Evaluation

6.1 Current Technical Results

6.1.1 Dataset and Training Summary

6.1.2 Qwen3.5 8-Fact V1 Baseline

6.1.3 Qwen3.5 13-Fact V2 Results

6.1.4 Gemma-4 13-Fact V2 Comparison

6.1.5 Error Analysis

6.2 Planned Runtime Data Collection and Human-Subject Evaluation

6.2.1 Original Evaluation Vision

6.2.2 Planned Evaluation Metrics

6.2.3 Required Runtime Data Collection Upgrades

6.2.4 Planned Experiment Design

6.2.5 Known Threats to Validity

7 Discussion

7.1 Why the Project Shifted from VR-First to Telemetry/VLM-Heavy

7.2 What Current Benchmarks Do and Do Not Prove

7.3 Ontology Evolution: 8-Fact to 13-Fact

7.4 Why Human-Subject Evidence Remains Essential

7.5 Risks in Interpreting Benchmark Success as Learning Success

7.6 Limitations

8 Conclusion

8.1 Summary of Completed Work

8.2 Future Work

References

9 Appendix